

CHAPTER 6

Verilog 電路設計 的基本觀念



在本章將為各位介紹：Verilog 的訊號 (signal) 與變數 (variable) 觀念、使用括弧來描述複雜的電路結構、善用「常數」 (Constant)、運算元的位用寬度 (operator bitwidth) 以及重置 (Reset) 訊號與預設 (Preset) 訊號的重要性。



6.1 訊號 (signal) 與變數 (variable)

在標準的 C 或 Pascal 語言裏頭，程式的執行是按照程式碼編寫的先後順序一個接一個有既定順序 (Sequential) 地被執行。

基本上，在同一時間中只有一個程式碼被執行。但在真實的硬體世界裏，許多彼此不相關連的訊號變化或運算可以平行進行 (Concurrent)，為了能充份表現硬體線路的所有特性，Verilog 允許 Sequential 和 Concurrent 兩種不同的描述存在。

在 module 的主區塊中只能接受 Concurrent 的敘述，但在 always 的區塊裏頭，所有的敘述 (statement) 則是循序地被執行，是所謂的 Sequential。

同樣的電路往往有多種不同的表示法，如何選擇最有效率的描述，是 Verilog 設計者極為重要的一項課題。

Verilog 則是使用 `<=` 號來代表『訊號』 (signal, Non-Blocking) 的資料指定，使用 `=` 符號來代表『變數』 (variable, Blocking) 的資料指定。

→ Sequential

↳ Concurrent

『訊號』 (signal) 是 Concurrent 的敘述，而『變數』 (variable) 則是 Sequential 的敘述。

注意
NOTICE

VHDL 是使用 `<=` 符號來代表『訊號』(signal, Non-Blocking) 的資料指定，使用 `:=` 符號來代表『變數』(variable, Blocking) 的資料指定。二者容易混淆，請不要弄錯了！

底下先使用表格化的方式來區分：『訊號』(signal, Non-Blocking) 與『變數』(variable, Blocking) 的差異，之後會一一用文字來解說，並用淺顯易懂的 Verilog 例子來說明。

Verilog	『訊號』 signal(<=)	『變數』 variable(=)
VHDL	『訊號』 signal(<=)	『變數』 variable(:=)
指定方式	訊號名稱 <= 指定運算/值;	變數名稱 := 指定運算/值;
功 用	全域性儲存資料的地方 以及電路的連線，連接 (wire, circuit interconnect)	區域性／暫時性 儲存資料的地方 (local storage)
有效範圍	全域性的 可以讓單一 module 內的不同的 always 存取的到	區域性的 只能用在單一的 always 中
動作方式	寫在 always 區塊之內訊號(signal) 的值之更新，需要等到該 always 所有的變數以及訊號更新了之後才開始去更新，而且本次被更新的值並不能立即再被其他變數或者是訊號拿來使用，取用它的話，會是得到本次被更新之前的值。	變數(variable) 的值會立即被更新，並且最新被更新的值可以立即再被其他訊號或變數拿來使用。 類似一般程式語言的變數指定動作。

6.1.1 訊號 (signal)

寫在 always 區塊之內訊號(signal) 的值之更新需要等到該 always 所有的變數以及訊號更新了之後才開始去更新，而且本次被更新的值並不能立即再被其他變數或者是訊號拿來使用。

- 訊號指定敘述(=) 寫在 always 區塊之外，稱之為同時性的訊號指定 (Concurrent signal assignment)。

➡ 訊號指定敘述 ($\leq$) 寫在 `always` 區塊之內，稱之為循序性的訊號指定 (Sequential signal assignment)。

差異在於寫在 `always` 區塊之內的訊號指定敘述 ($\leq$) 必須看 `always (sensitivity-list) ...begin...end` 中，只要在 `sensitivity-list` “感測列” 中的訊號有變化，那麼才會進到該 `always` 區塊中去處理 `always` 內的動作。例如：

```
Data1 <= DataB; // 只要 DataB 有變化，DataB 就給 Data1
always @( clk ) // 必須等到 clk 有變化
begin
    Data2 <= DataB; // DataB 才會給 Data2
end
```

如果在 `always` 中把值或者是運算式給某個變數的話，最後的那一個給值或者是運算式的動作為最終的結果。例如：

```
always @( Clk )
begin
    XS1 <= Data4 * Data5; // 這一行終將沒有作用的
    S1 <= Data1 * Data2;

    S2 <= S1 * Data3;
    Result <= S2;
end;
```

聰明的您一定看得出來：Result 最終會等於 `Data1 * Data2 * Data3`，而不是 `Data4 * Data5 * Data3`。

如果二個及以上的指定敘述把值或者是運算完的結果要給**同一個訊號**的話，那麼該訊號的值會是什麼值呢？

合成出來的電路是由最終訊號 (signal) 的值或是運算式，以及合成後將燒錄到那種製程的元件上 (target technology) 來決定的。最終訊號 (signal) 的邏輯值可能是無效的、wired AND、wired OR 或者是 tri-state。

例如：

```
reg out1, out2;

always @( Clk )
begin
    out1 <= a;
    out2 <= out1;
end;

always @( Clk )
begin
    out1 <= b;
    out2 <= out1;
end;
```

out2 最終會等於 **b**，因為 **out1** 的型態是 **reg**，先出現 **out1 <= a;**，後出現 **out1 <= b;**，因此 **out1 <= a;** 這一行會被合成軟體給忽略掉。

例如：

```
wire out;

assign out = a;
assign out = b;
assign out = c;
```

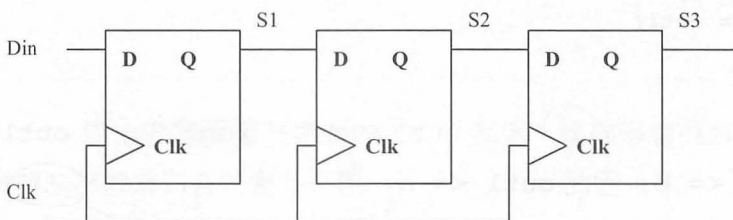
out 最終會等於 **c**，即 **assign out = c;**，因為 **out** 的型態是 **wire**，**assign out = a;** 以及 **assign out = b;** 這二行會被合成軟體給忽略掉。

不過如果 **out** 的型態是 **wand** 或 **wor**，**assign out = a;** 與 **assign out = b;** 這二行就不會被合成軟體給忽略掉。**out** 的值會是什麼請您參閱 3.2.3 **wand**(Wired-AND 型接線)，**wor**(Wired-OR 型接線) 敘述。

又例如：

```
reg Din, S1, S2, S3;
always @(posedge Clk)
begin
    if (Clk)
    begin
        S1 <= Din;
        S2 <= S1;
        S3 <= S2;
    end
end
end
```

要經過 3 個 Clk 之後，S3 才會等於 Din。



<= 可以將在一個 Clk 時間內無法完成的複雜運算、切割成 2~數級。

■ 優點：讓整體運算速度提昇。

■ 缺點：運算輸出的時因而延遲 1~(數級-1) 個 Clk。

例如：

```
always @( Clk )
begin
    Data3_Delay1 <= Data3;           // Data3 延遲 1 個 Clk
    Data4_Delay1 <= Data4;           // Data4 延遲 1 個 Clk
    Data4_Delay2 <= Data4_Delay1;    // Data4 延遲 2 個 Clk
end;

always @( Clk )
begin
    R1 <= Data1 * Data2;
    R2 <= R1 * Data3_Delay1;         // 乘以 Data3 延遲 1 個 Clk
    R3 <= R2 * Data4_Delay2;         // 乘以 Data4 延遲 2 個 Clk
    Result <= R3;
end;
```

聰明的您一定看得出來：Result 最終會等於 $\text{Data1} * \text{Data2} * \text{Data3} * \text{Data4}$ 。但是請想想看，為什麼 $R2 = R1$ 乘以 (**Data3_Delay1**, Data3 延遲 1 個 Clk)、 $R3 = R2$ 乘以 (**Data4_Delay2**, Data4 延遲 2 個 Clk) 呢？

6.1.2 把數值／運算式指定給訊號 (signal) 或變數 (variable)

(一) 對“整個“變數 (all bits) 給數值／運算式

以 `reg [7:0] byte;` 為例：

```
byte <= 8'b01010101; // 85

reg [6:0] Data_1;
reg [5:0] Data_2;

Data_1 <= 7'b0010101;
Data_2 <= 6'b010101;
byte <= {1'b0, Data_1} + Data_2;
```

(二) 對變數中“某個位元” (single bit) 給數值／運算式

```
byte[7] <= 1b0; byte[6] <= 1b1; byte[5] <= 1b0; byte[4] <= 1b1;
byte[3] <= 1b0; byte[2] <= 1b1; byte[1] <= 1b0; byte[0] <= 1b1;
byte[0] <= a | b;
```

(三) 對變數中“某些位元範圍” (Bit-slicing) 給數值／運算式

```
byte[7:4] <= 4'b0101; byte[3:0] <= 4'b0101;
byte[7:4] <= {a, b, c, d};
```

6.1.3 變數 (variable)

Verilog 是使用 = 符號來代表 variable (Blocking) 的動作。

變數的值會立即被更新，並且最新被更新的值可以立即再被其他訊號或變數拿來使用。

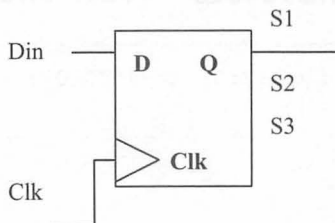
那麼要用什麼方法才能設定 <初始值> 呢？

對於“暫存器”(Register)、“正反器”(flip-flops)之類的元件就在您的電路中加入與時脈(Clock)訊號同步(Synchronous)或者是非同步(Asynchronous)的 Reset(重置)、Clear(清除)、Set(設定)或者是 Preset(預設)之類的輸入訊號來控制它們，讓它們在特定的時間被設定 <初始值>。

例如：

```
reg Din, S1, S2, S3;
always @( Clk )
begin
    if (Clk)
    begin
        S1 = Din;
        S2 = S1;
        S3 <= S2;
    end
end
end
```

只要經過 1 個 Clk 之後，S3 就會等於 Din。

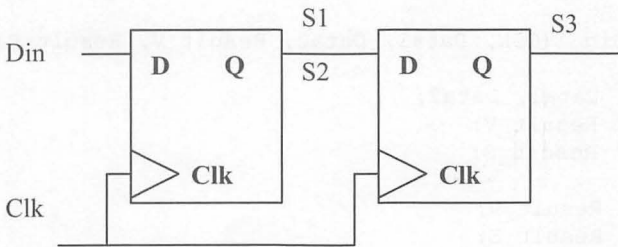


請和之前那個用 signal 訊號的寫法，要經過 3 個 Clk 之後，S3 才會等於 Din 的寫法相比較，了解其中的差異。

又例如：

```
reg Din, S1, S2, S3;
always @( Clk )
begin
    if (Clk)
    begin
        S1 <= Din;
        S2 = S1;
        S3 <= S2;
    end
end
end
```

要經過 2 個 Clk 之後，S3 才會等於 Din。



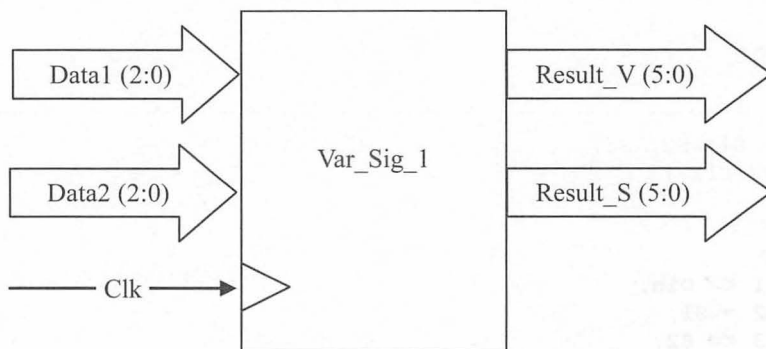
請和之前的 2 種寫法互相比較，了解其中的差異。



6.2 always 中的訊號 (signal) 與變數 (variable)

看看底下這個 signal 與 variable 互相對照比較的例子，再來看它們的模擬結果。

6.2.1 訊號與變數，範例 1



▲ 訊號與變數，範例 1 的方塊圖

【訊號與變數，範例 1 的 Verilog 程式碼：var_sig_1.v】

```

// Var_Sig_1.V
module Var_Sig_1(Clk, Data1, Data2, Result_V, Result_S);
input Clk;
input [2:0] Data1, Data2;
output [5:0] Result_V;
output [5:0] Result_S;

wire [5:0] Result_V;
wire [5:0] Result_S;

reg [5:0] V1_D1_m_D2;
reg [5:0] V2_D1_m_D2;

reg [5:0] S1_D1_m_D2;
reg [5:0] S2_D1_m_D2;

// variable (block)
always @(posedge Clk)
begin
    if (Clk)
    begin
        V1_D1_m_D2 = Data1 * Data2;
        V2_D1_m_D2 = V1_D1_m_D2;
    end
end
end
  
```

```

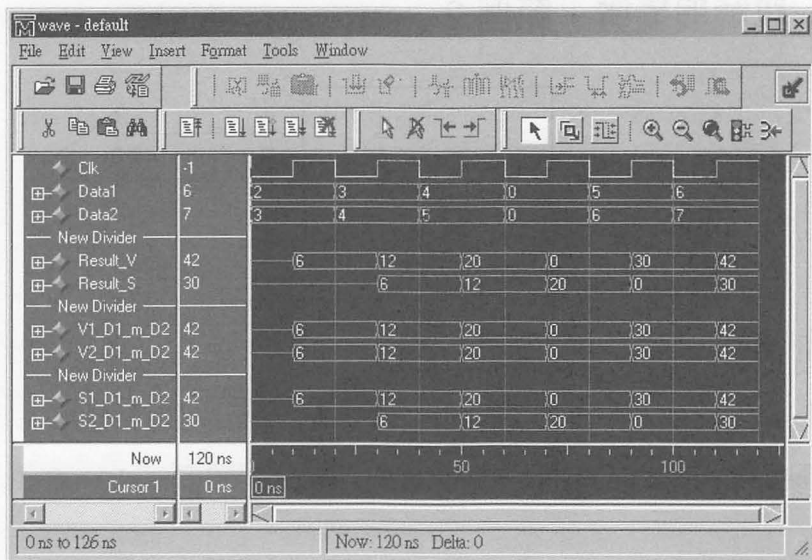
assign Result_V = V2_D1_m_D2;

// signal (non-block)
always @(posedge Clk)
begin
    if (Clk)
    begin
        S1_D1_m_D2 <= Data1 * Data2;
        S2_D1_m_D2 <= S1_D1_m_D2;
    end
end

assign Result_S = S2_D1_m_D2;

endmodule

```



▲ var_sig_1 的模擬結果

var_sig_1 模擬結果說明

用 **variable** 的寫法告， $V_D1_m_D2 = Data1 * Data2$ ；會立即執行，用 **assign** 輸出訊號 $Result_V = V_D1_m_D2$ 也會就等於 $Data1 * Data2$ 。

用 **signal** 的寫法告，`S1_D1_m_D2 <= Data1 * Data2;` 並不會立即執行，用 `assign` 輸出訊號 `Result_S <= S2_D1_m_D2` 會就等於上一次 `Data1 * Data2` 運算完的值，所以要讓 `Result_S` 等於 `Data1 * Data2` 可得要多等到在 `always @(posedge Clk)` 括弧中 Sensitivity List 裡頭的 `Clk` 值有由 '0' → '1' 的變化，才會有上一次 `Data1 * Data2` 運算完的值，如果 `always @(posedge Clk)` 改寫成 `always @(Data1, Data2)` 那麼就得等到 **Data1** 或 **Data2** 有變化，`Result_S` 會有上一次 `Data1 * Data2` 運算完的值。

6.2.2 訊號與變數，範例 2



▲ 訊號與變數，範例 2 的方塊圖

【訊號與變數，範例 2 的 Verilog 程式碼：var_sig_2.v】

```
// Var_Sig_2.V
module Var_Sig_2(DataIn, DataOut_s1, DataOut_s2);
input  [2:0] DataIn;
output [2:0] DataOut_s1;
output [2:0] DataOut_s2;

wire [2:0] DataOut_s1;
wire [2:0] DataOut_s2;

reg [2:0] sDataOut_s1;
reg [2:0] sDataOut_s2;
```

```

wire      c1, c2;
reg       s1, s2;
reg       v1, v2;

assign c1 = DataIn[0] & DataIn[1]; // concurrent
assign c2 = c1; // concurrent

always @(DataIn)
begin
    sDataOut_s1 <= DataIn;    // signal (non-block)
    sDataOut_s2 <= sDataOut_s1; // signal (non-block)

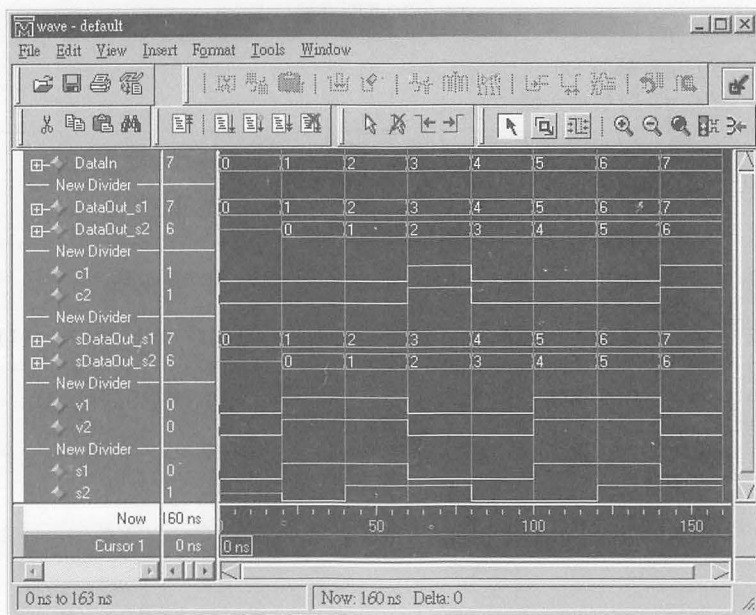
    v1 = DataIn[0] ^ DataIn[1]; // variable (block)
    v2 = v1;                    // variable (block)

    s1 <= v1; // signal (non-block)
    s2 <= s1; // signal (non-block)
end

assign DataOut_s1 = sDataOut_s1; // concurrent
assign DataOut_s2 = sDataOut_s2; // concurrent

endmodule

```



▲ var_sig_2 的模擬結果

var_sig_2 模擬結果說明

在 `always @(DataIn)` 內的變數(variable) `v1` 與 `v2` 指定動作 `=` 均為 Sequential Statement (循序動作敘述)，該動作會立即被處理，並且最新被更新的值可以立即再被其他訊號或變數拿來使用。

在 `always @(DataIn)` 內的訊號(signal) `s1` 與 `s2` 指定動作 `<=` 均為 Concurrent Statement (同時動作敘述)，因為是同時動作所以就有先後順序上的關係。

在 `always @(DataIn)` 內的動作情形：在 `always @(DataIn)` 表示當 `DataIn` 值有改變，就會做底下的動作。

```
sDataOut_s1 <= DataIn;           // sDataOut_s1 等於 DataIn
sDataOut_s2 <= sDataOut_s1;      // sDataOut_s1 等於前一個 DataIn

v1 = DataIn[0] ^ DataIn[1];      // v1 立即等於 DataIn[0] ^ DataIn[1]
v2 = v1;                        // v2 立即等於 v1

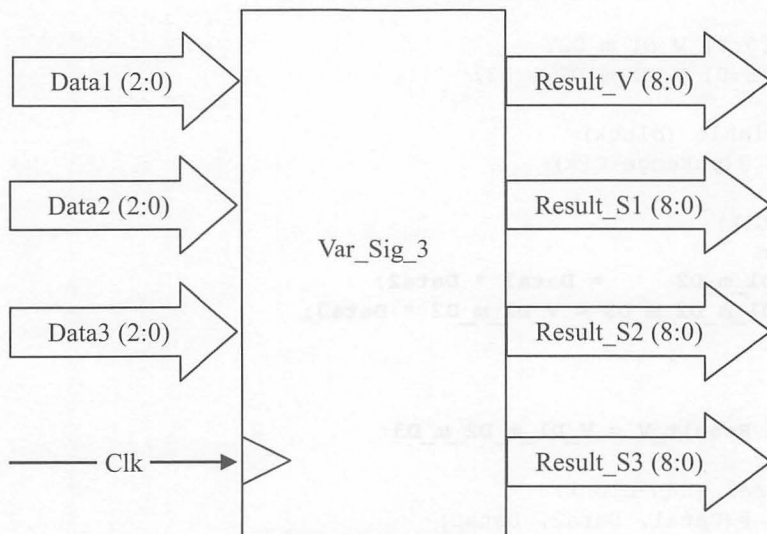
s1 <= v1; // s1 等於 v1
s2 <= s1; // s2 等於前一個 v1
```

不在 `always @(DataIn)` 內的動作情形：

```
assign c1 = DataIn[0] & DataIn[1]; // c1 等於 DataIn(0) and
                                   DataIn(1)
assign c2 = c1;                   // c2 也是等於 DataIn(0) and DataIn(1)

assign DataOut_s1 = sDataOut_s1; // 立即等於 sDataOut_s1
assign DataOut_s2 = sDataOut_s2; // 立即等於 sDataOut_s2
```

6.2.3 訊號與變數，範例 3



▲ 訊號與變數，範例 3 的方塊圖

【訊號與變數，範例 3 的 Verilog 程式碼：var_sig_3.v】

```
// Var_Sig_3.V
module Var_Sig_3(Clk, Data1, Data2, Data3, Result_V, Result_S1,
Result_S2, Result_S3);
input  Clk;
input  [2:0] Data1, Data2, Data3;
output [8:0] Result_V;
output [8:0] Result_S1;
output [8:0] Result_S2;
output [8:0] Result_S3;

wire [8:0] Result_V;
wire [8:0] Result_S1;
wire [8:0] Result_S2;
wire [8:0] Result_S3;

reg [5:0] S1_D1_m_D2;
reg [8:0] S1_D1_m_D2_m_D3;
reg [5:0] S2_D1_m_D2;
```

```

reg      [8:0] S2_D1_m_D2_m_D3;
wire     [5:0] S3_D1_m_D2;
wire     [8:0] S3_D1_m_D2_m_D3;

reg      [5:0] V_D1_m_D2;
reg      [8:0] V_D1_m_D2_m_D3;

// variable (block)
always @(posedge Clk)
begin
    if (Clk)
    begin
        V_D1_m_D2      = Data1 * Data2;
        V_D1_m_D2_m_D3 = V_D1_m_D2 * Data3;
    end
end

assign Result_V = V_D1_m_D2_m_D3;

// signal (non-block)
always @(Data1, Data2, Data3)
begin
    S1_D1_m_D2      <= Data1      * Data2;
    S1_D1_m_D2_m_D3 <= S1_D1_m_D2 * Data3;
end
assign Result_S1 = S1_D1_m_D2_m_D3;

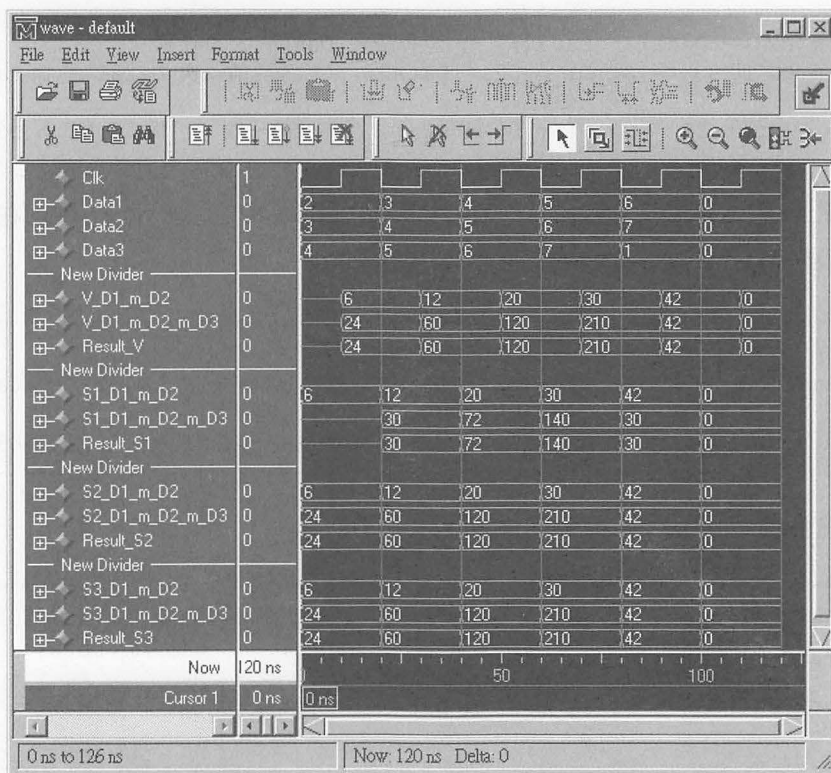
// variable (block)
always @(Data1, Data2, Data3)
begin
    S2_D1_m_D2      = Data1      * Data2;
    S2_D1_m_D2_m_D3 <= S2_D1_m_D2 * Data3;
end

assign Result_S2 = S2_D1_m_D2_m_D3;

// concurrent
assign S3_D1_m_D2      = Data1      * Data2;
assign S3_D1_m_D2_m_D3 = S3_D1_m_D2 * Data3;
assign Result_S3      = S3_D1_m_D2_m_D3;

endmodule

```



▲ var_sig_3 的模擬結果

var_sig_3 模擬結果說明

在第 1 個 `always @(posedge Clk)` 內，`if(Clk)` 表示當 Clk 在上緣觸發(由 0 變 1)的時候，就會做底下的動作：

```
V_D1_m_D2 = Data1 * Data2; // 立即等於 Data1 * Data2
V_D1_m_D2_m_D3 = V_D1_m_D2 * Data3; // 立即等於 Data1 * Data2 *
Data3
assign Result_V = V_D1_m_D2_m_D3; // 立即等於 Data1 * Data2 * Data3
```

在 `always @(Data1, Data2, Data3)` 表示當 Data1 或 Data2 或 Data3 其中一個值有改變，就會做底下的動作：

```
S1_D1_m_D2 <= Data1 * Data2; // 立即等於 Data1 * Data2
S1_D1_m_D2_m_D3 <= S1_D1_m_D2 * Data3; // 等於前一次 Data1 * Data2
                                     再 * Data3
assign Result_S1 = S1_D1_m_D2_m_D3; // 等於前一次 S1_D1_m_D2_m_D3
```

在第 3 個 always @(posedge Clk) 內，if(Clk) 表示當 Clk 在上緣觸發(由 0 變 1)的時候，就會做底下的動作：

```
S2_D1_m_D2 = Data1 * Data2; // 立即等於 Data1 * Data2
S2_D1_m_D2_m_D3 <= S2_D1_m_D2 * Data3; // 等於前一次 Data1 * Data2
                                     再 * Data3
assign Result_S1 = S1_D1_m_D2_m_D3; // 等於前一次 S1_D1_m_D2_m_D3
```

不在任何一個 process 內的是做底下的動作：

```
assign S3_D1_m_D2 = Data1 * Data2; // 立即等於 Data1 * Data2
assign S3_D1_m_D2_m_D3 = S3_D1_m_D2 * Data3; // 立即等於 Data1 *
                                     Data2 * Data3
assign Result_S3 = S3_D1_m_D2_m_D3; // 立即等於 Data1 * Data2 *
                                     Data3
```

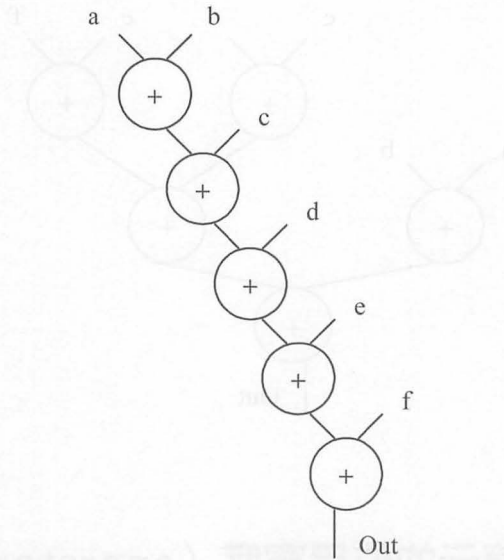


6.3 使用括弧來描述複雜的電路結構

在底下的範例中：例 3 使用括弧的技巧，考慮了：訊號到達的時間(data arrival time)及訊號相依(data dependency)的關係，來得到功能相同；而且使用一樣多個加法電路，但因為加法動作的層數不同，所以例 2 合成出來的電路速度會比例 1 還要快，而例 3 合成出來的電路速度會比例 2 還要快。

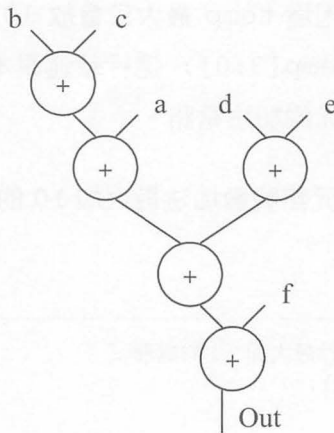
例 1 Out <= a + b + c + d + e + f;

5 層加法動作之後，才會得到結果 Out。



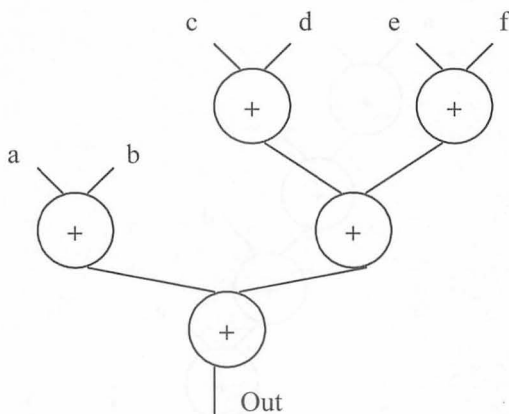
例 2 `Out <= ((a + (b + c)) + (d + e)) + f;`

4 層加法動作之後，才會得到結果 Out。



例 3 `Out <= (a + b) + ((c + d) + (e + f));`

3 層加法動作後，就可以得到結果 Out。



6.4 運算元的位用寬度 (operator bitwidth)

電路中沒有用到的部份儘量不要去使用它，就可以因此減少不必要的接線數目。

以底下這個例子來說，因為 temp 最大只會放 10，所以只需 4 位元就夠用了，`Add_LT_10 = a + temp[3:0]`；這行敘述原本需要由 8 個位元的加法電路，改寫後只需要 4 個位元的加法電路。

範例 使用函數設計 8 位元無號數加法最大加 10 的處理

【程式碼 Add_LT10.V】

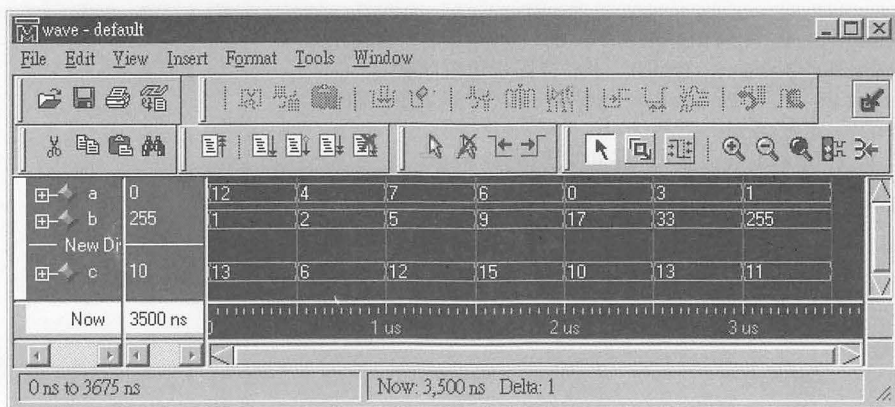
```
// Add_LT10.V, 使用函數設計最大加 10 的處理
module Add_LT10(a, b, c);
input    [7:0] a, b;
output   [8:0] c;
wire     [8:0] c;

    assign c = Add_LT_10(a, b);
```

```

function  [8:0] Add_LT_10;
input     [7:0] a, b;
reg       [7:0] temp;
begin
    if (b < 10)
        temp = b;
    else
        temp = 10; // 因為 temp 最大只會放 10, 4 位元就夠用了
    Add_LT_10 = a + temp[3:0]; // temp[3:0] 可存數值範圍: 0 ~ 15.
    // 因 temp 必小於等於 10, 所以只要使用 temp 的 [3:0] 就可以了.
end
endfunction
endmodule

```



▲ Add_Less_Than10_Unsigned 的模擬結果



6.5 重置 (Reset) 訊號與預設 (Preset) 訊號的重要性

電路重置 (Reset) 訊號的重要性，包括：

- 電路模擬比較容易。
- 電路除錯比較容易。

- 有重置 (Reset) 訊號，合成出來的電路動作情形和電路模擬時的結果才會相同。
- 少了重置 (Reset) 訊號，無法保證當電路上來之後，暫存器的值以及有限狀態機的狀態值會是什麼值，不一定全是 '0'，也不一定全是 '1'。
- 如果不想用重置 (Reset) 訊號來為暫存器設定初始值，也可以用預設 (Preset) 訊號來為暫存器設定某個值，功能是一樣的。

例如：

```
signal count : integer := 0; // 0 不要寫在這
always @( Reset, Inc ) // 非同步重置訊號
// always @ ( Preset, Inc ) // 非同步預設訊號
begin
    if ( Reset ) // 非同步重置
        // if (Preset ) // 非同步預設，只是預設的值是 0 而已
        count = 0; // 寫在這就對了
    else
        if ( Inc )
            count = count + 1;
end
```

本章練習

Question 1：

訊號 (signal) 與變數 (variable) 的使用時機分別是？請舉例說明。

Question 2：

訊號重置 (Reset) 與訊號預設 (Preset) 的差異又是？請舉例說明。

PS Google 可以幫您找到解答。